From FPGA to Vector Drawing on an Oscilloscope

Verified experiment • 2 digital pins • 2 resistors • 2 capacitors • XY mode

How does an 8-bit number become the position of a luminous spot?

COORDINATES → PULSE DENSITY → RC FILTER → VOLTAGE → XY POSITION

Demonstration: P • A • U • L • ♥, S1/S2 control and sound feedback

Learning objectives

- Why an oscilloscope can draw without a dedicated DAC.
- How an (X,Y) point is represented using 8-bit values.
- Why the order of points determines the visible trajectory.
- How an accumulator generates pulses with controlled density.
- Why the R–C network produces the required analog voltage.
- How S1/S2 and the buzzer are integrated into synchronous logic.

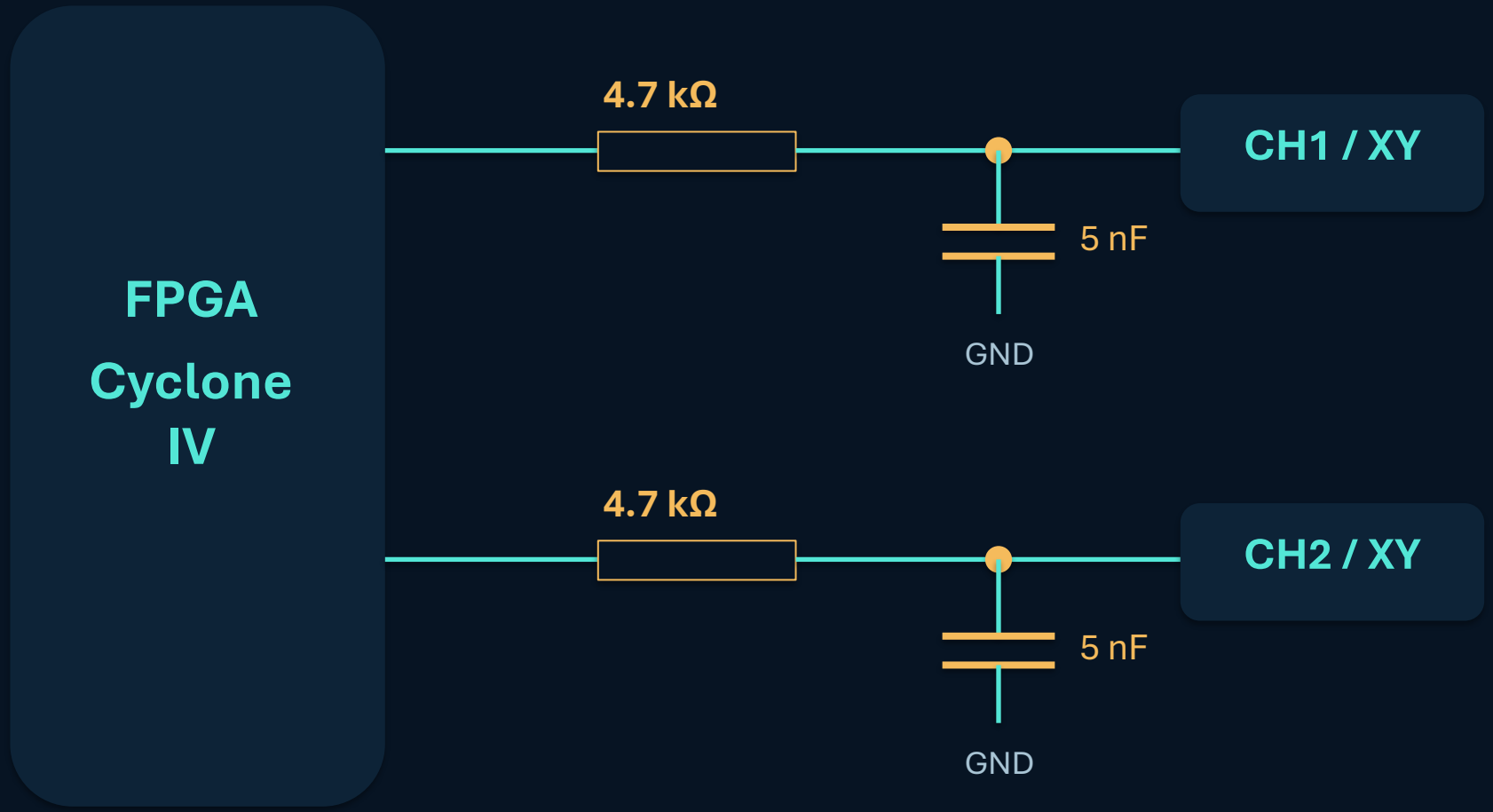
Complete system: functional blocks



The same architecture operates for X and Y at 50 MHz.

Actual circuit schematic

Each axis uses one FPGA pin, one resistor, and one capacitor.

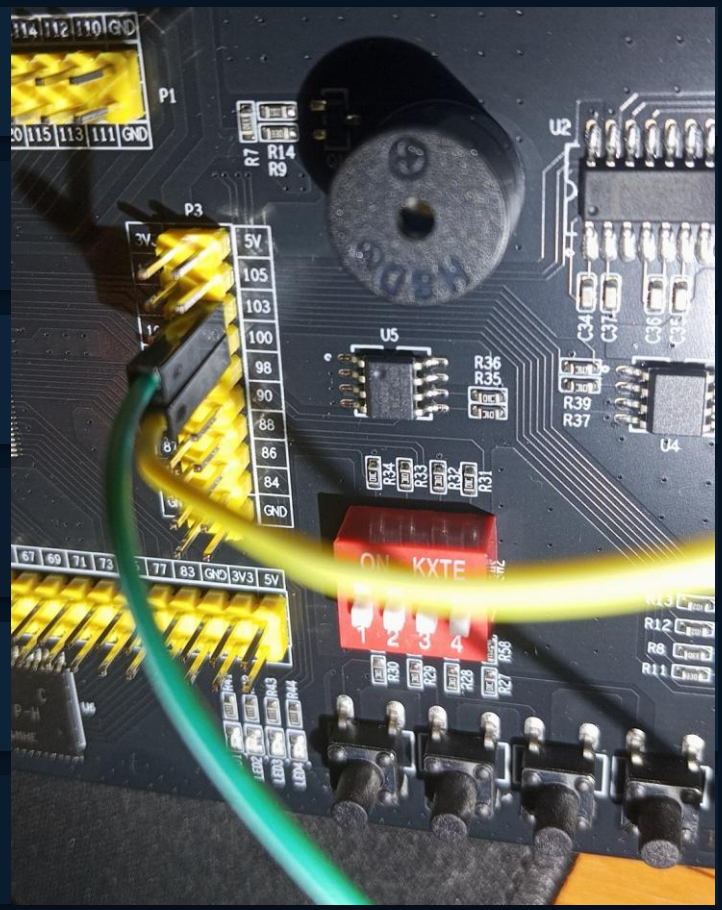




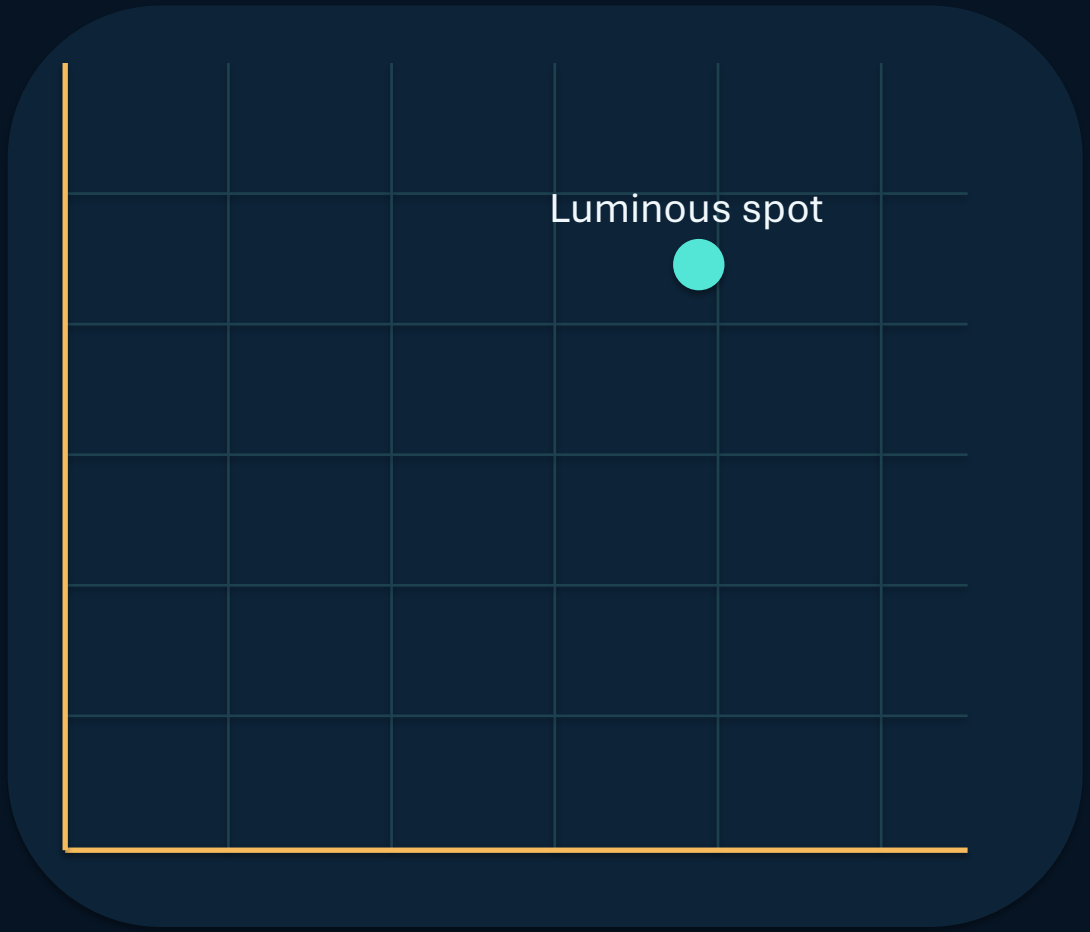
FPGA and oscilloscope grounds must be connected together.

Pin assignments – OMDAZZ Cyclone IV

clk	PIN_23	50 MHz clock
X	PIN_103	Horizontal output → RC → CH1
Y	PIN_100	Vertical output → RC → CH2
S1 / KEY1	PIN_88	Next character
S2 / KEY2	PIN_89	Previous character
buzzer	PIN_110	Beep on press; idle HIGH



XY mode: voltages move the spot



- CH1 controls X (horizontal).
- CH2 controls Y (vertical).
- Voltages are smoothed after the RC filter.
- These are analog positions, not display pixels.

8-bit registers: coordinate range

```
reg [7:0] x;    reg [7:0] y;
```

- 8 bits ⇒ 2^8 = 256 possible values per axis.
- Minimum: 0. Maximum: 255.
- (128,128) is approximately the midpoint of the range.
- 256 × 256 = 65,536 possible digital coordinate pairs.

What does 8'd130 mean?

```
8'd130
||  └── 130: numeric value
|└───── d: decimal (base 10)
└────── 8: number of bits

8'd130 = 8'b10000010 = 8'h82
```

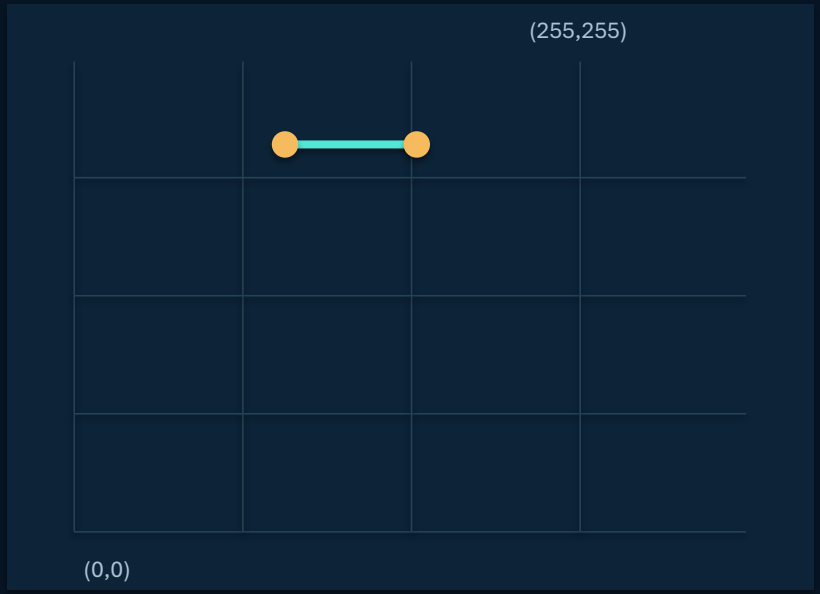
Note: 8 specifies the bit width; it is not multiplication.

Four values define a line segment

```
shape = {8'd80, 8'd210, 8'd130, 8'd210};

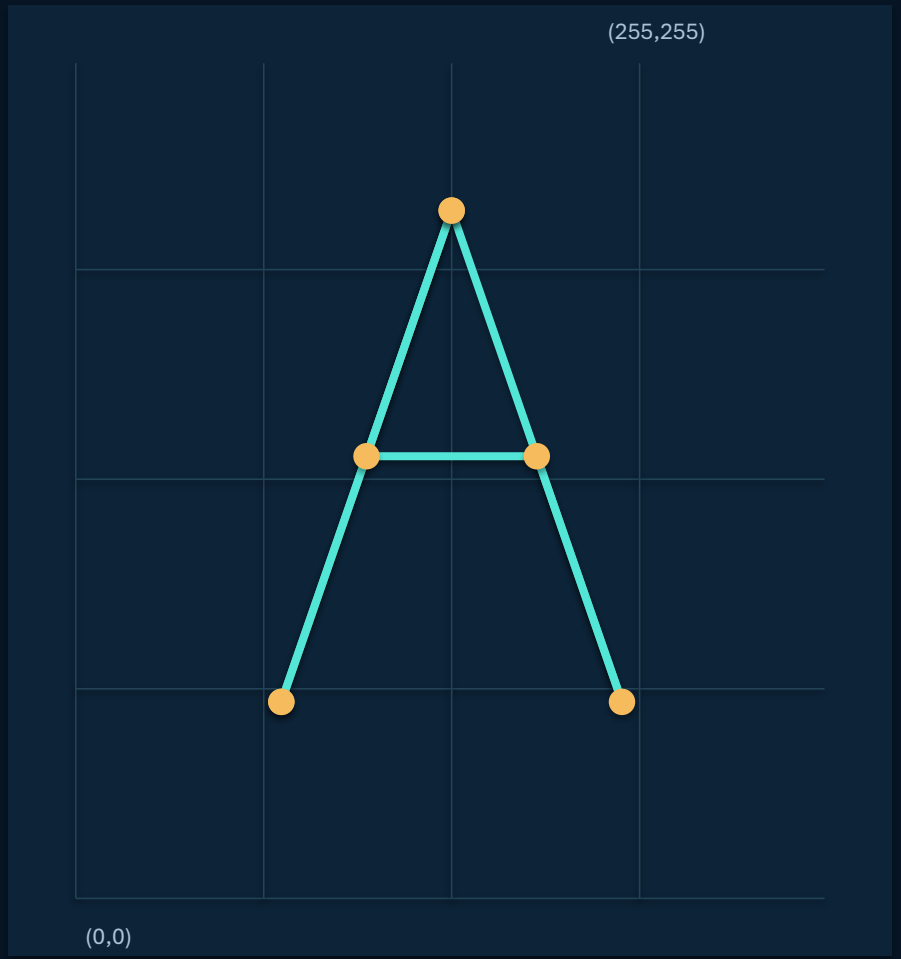
        { x0=80, y0=210, x1=130, y1=210 }
```

- Starting point: A = (80, 210).
- Endpoint: B = (130, 210).
- The same Y value ⇒ a horizontal segment.



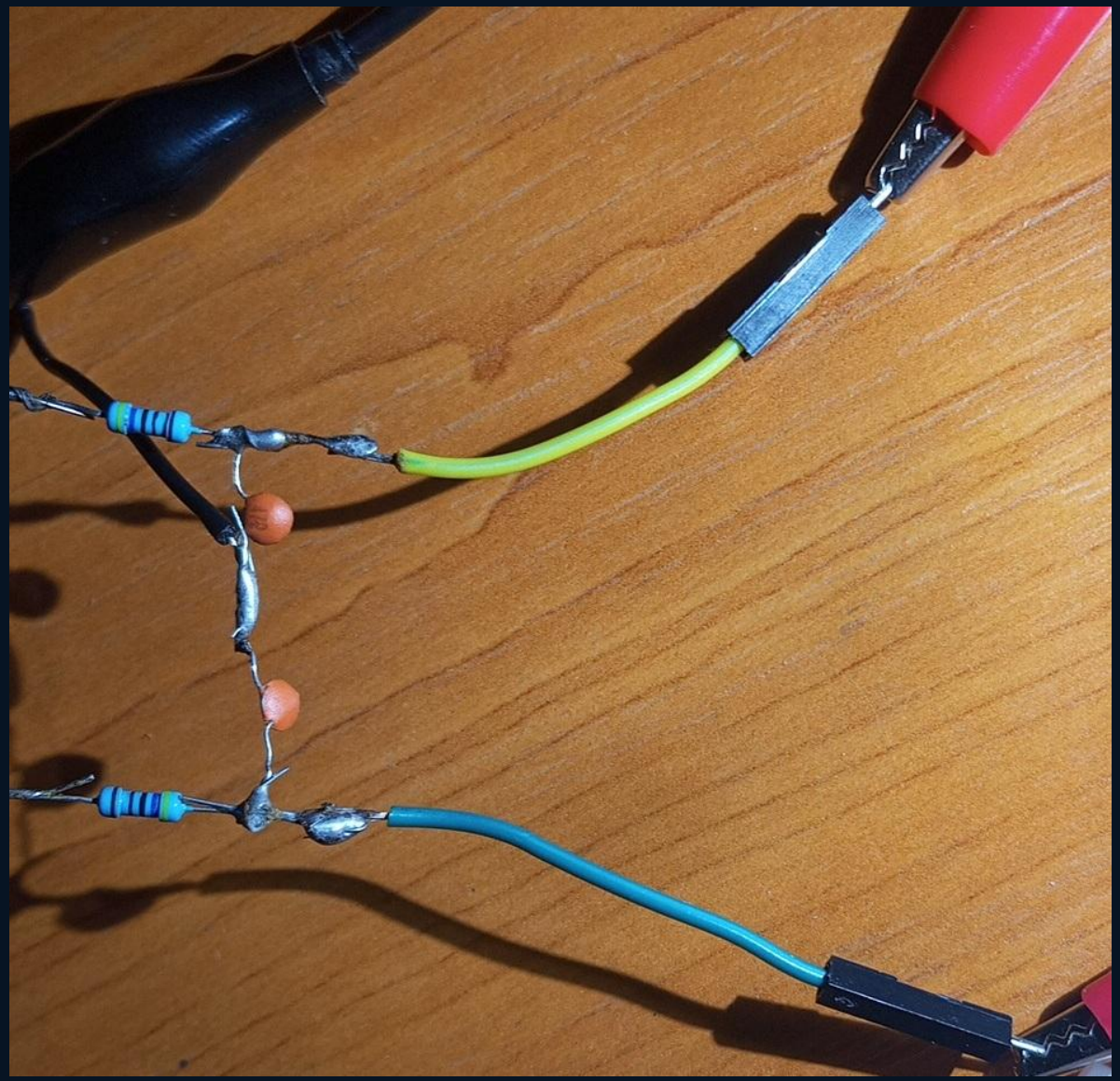
Why must coordinates be consecutive?

- One segment joins two consecutive points.
- The next segment starts where the previous one ends.
- The spot therefore moves without discontinuous jumps.
- Retracing also closes open paths (U, L), avoiding unwanted return lines without Z blanking.



Letter A: actual stroke order

1. (70,60) → (128,210): left foot → top
2. (128,210) → (186,60): top → right foot
3. (186,60) → (157,135): back up right stroke
4. (157,135) → (99,135): horizontal crossbar
5. (99,135) → (128,210): back up left stroke
6. (128,210) → (70,60): retrace left stroke



Interpolation: why not jump to the next point?

$$x(t) = x_0 + \frac{t(x_1 - x_0)}{63}$$

$$y(t) = y_0 + \frac{t(y_1 - y_0)}{63}$$

$$t = 0, 1, 2, \ldots, 63$$

- We compute 64 positions along each segment.
- X and Y advance proportionally, keeping the line straight.
- Without interpolation, independent axis increments can create unwanted corners.

Numerical example of interpolation

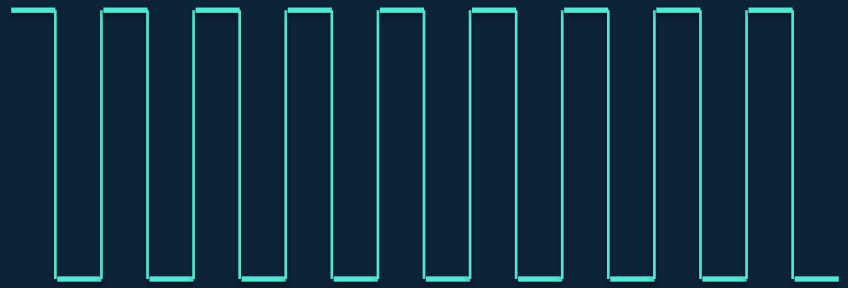
Segment: (80.210) → (130.210)

`t = 0`	X ≈ 80	Y = 210
`t = 16`	X ≈ 92	Y = 210
`t = 32`	X ≈ 105	Y = 210
`t = 48`	X ≈ 118	Y = 210
`t = 63`	X ≈ 130	Y = 210

Values are quantized to integers because x and y are 8-bit registers.

The FPGA does not directly generate 1.65 V!

DIGITAL PIN



Only approximately 0 V or 3.3 V

AFTER THE RC FILTER



Mean analog voltage ≈ 1.65 V

Essential code: sigma-delta modulator

```
reg [7:0] x = 80, y = 60;
reg [9:0] sx = 0, sy = 0;

always @(posedge clk) begin
    sx <= {2'b00, sx[7:0]} + {2'b00, x};
    sy <= {2'b00, sy[7:0]} + {2'b00, y};
end

assign X = sx[8];
assign Y = sy[8];
```

What exactly does the accumulator do?

- At each clock tick (20 ns), x is added to the low eight bits of sx.
- When the sum is ≥ 256, sx[8] becomes 1.
- The low bits retain the remainder modulo 256 for the next cycle.
- Larger x values produce a higher fraction of HIGH output cycles.
- x=0 ⇒ always LOW; x=128 ⇒ 50% HIGH; x=255 ⇒ almost always HIGH.

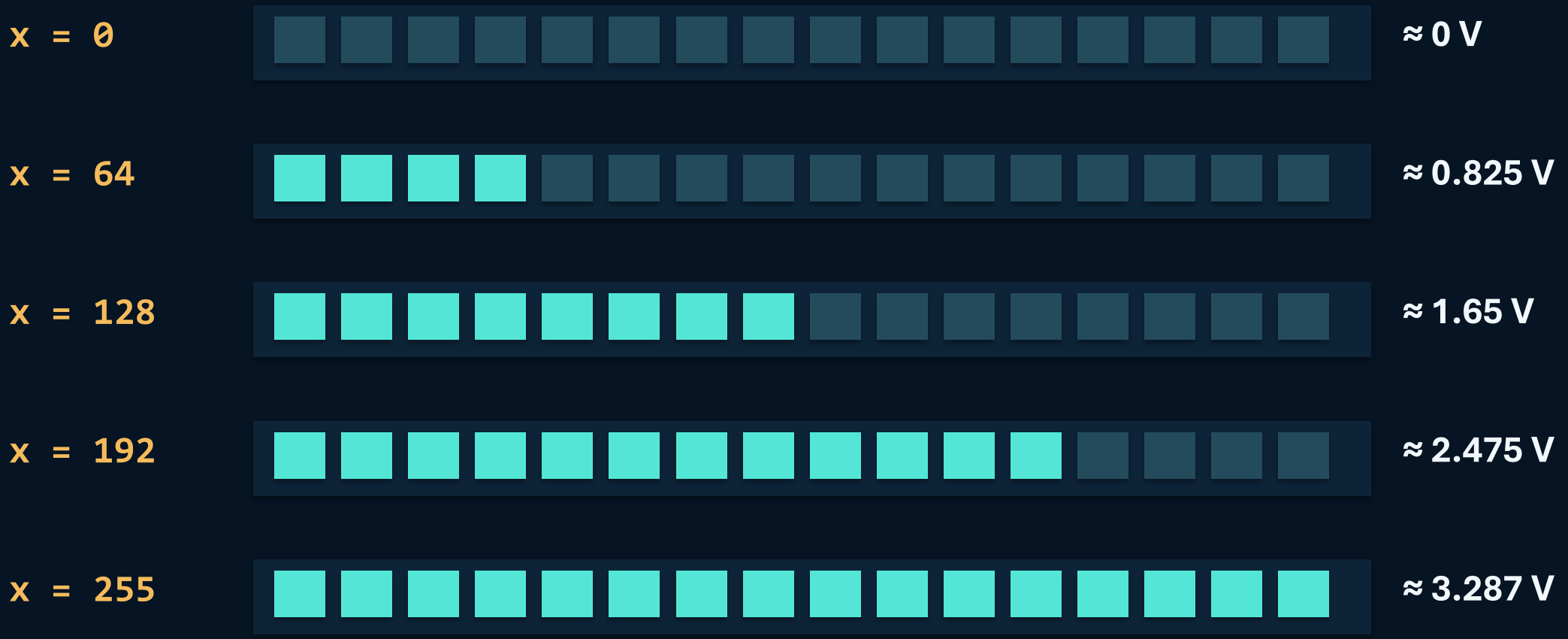
Step-by-step example: x = 64

64 = ¼ of 256 ⇒ the pin is HIGH about 25% of the time.

Cycle	previous remainder	remainder + 64	sx[8]
1	0	64	0
2	64	128	0
3	128	192	0
4	192	256	1
5	0	64	0
6	64	128	0

The sequence then repeats: 0, 0, 0, 1…

Pulse density controls voltage



Bars show the HIGH fraction, not the exact time sequence of pulses.

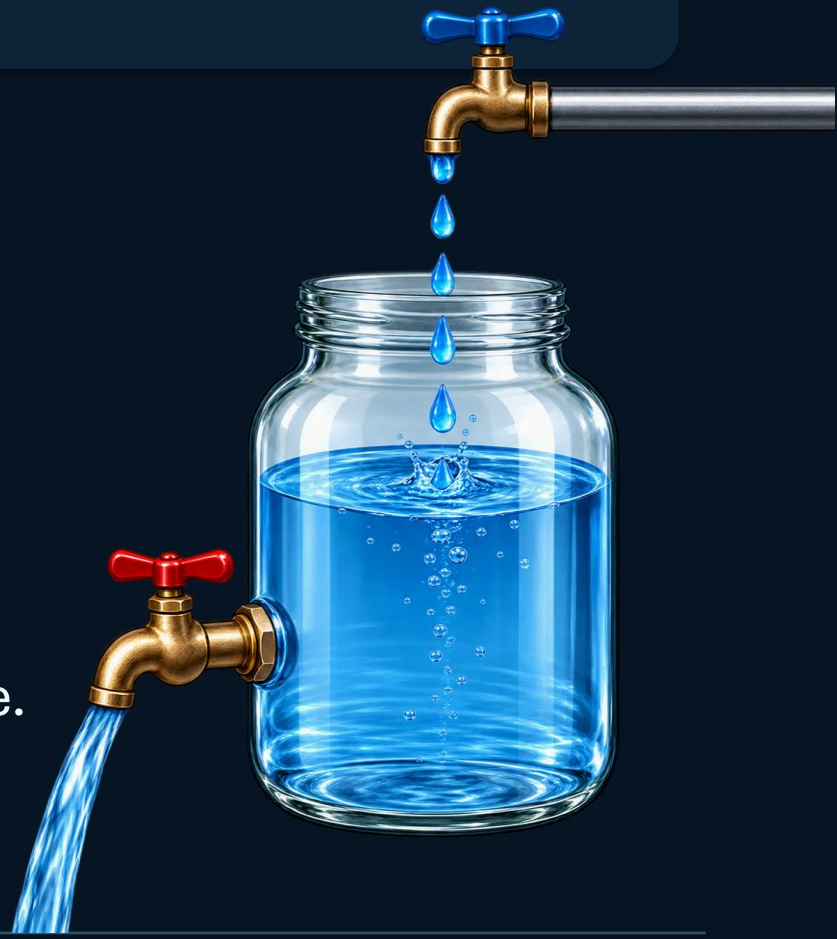
Why do we use a 4.7 kΩ resistor?

- It limits the instantaneous capacitor charge/discharge current.
- Together with the capacitor, it forms a low-pass filter.
- It isolates the high-speed digital pin from the oscilloscope's analog node.
- Too little resistance increases ripple and pin current; too much slows transitions.

Why do we use a 5000 pF capacitor?

$$Q = C \cdot V$$

- Pin HIGH: the capacitor charges through R.
- Pin LOW: the capacitor discharges through R.
- Rapid pulses establish an average charge level.
- A higher density of HIGH pulses ⇒ a higher average voltage.



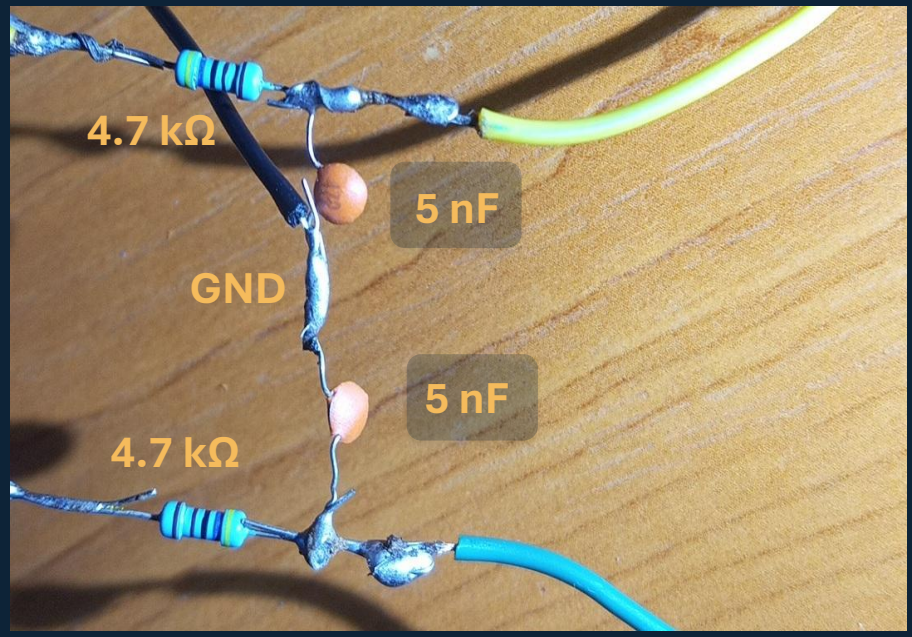
RC filter time constant

$$R = 4700\,\Omega$$

$$C = 5000\,pF = 5\,nF$$

$$\tau = RC = 23.5\,\mu s$$

$$f_c = \frac{1}{2\pi RC} \approx 6.77\,kHz$$



Trade-off: smooth fast pulses without slowing the trajectory excessively.

From coordinate to voltage after the RC filter

$$V_{avg} \approx \frac{V_{HIGH} \cdot value}{256}$$

$$V_{HIGH} = 3.3\,V$$

$$V_{avg} \approx \frac{3.3 \cdot value}{256}$$

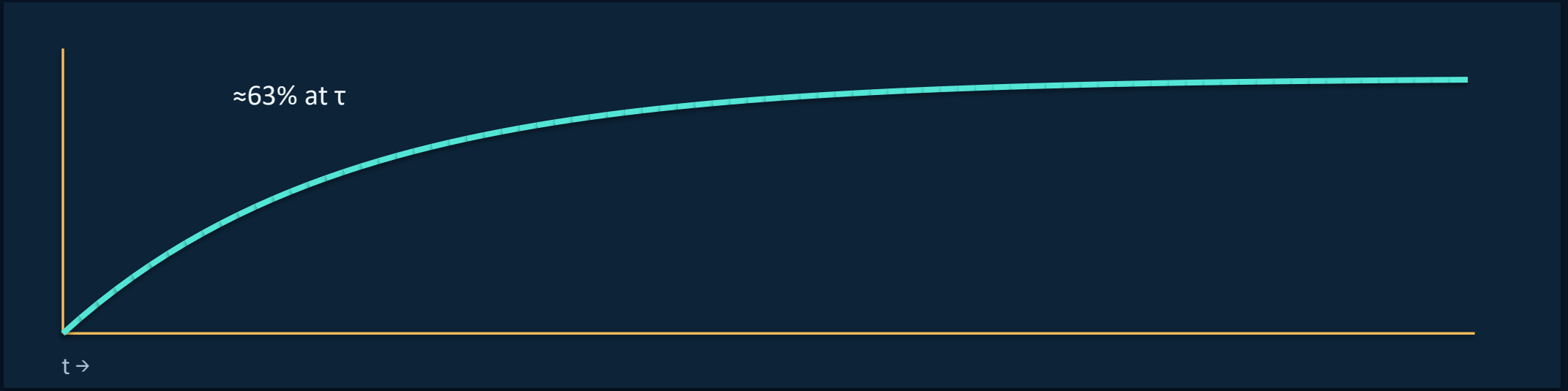
`|0 → 0 V| |64 → 0,825 V| |128 → 1,650 V| |192 → 2,475 V| |255 → 3,287 V|`

High-impedance approximation; VHIGH depends on the I/O standard and supply.

Why doesn't the voltage change instantly?

RC response to a voltage step:

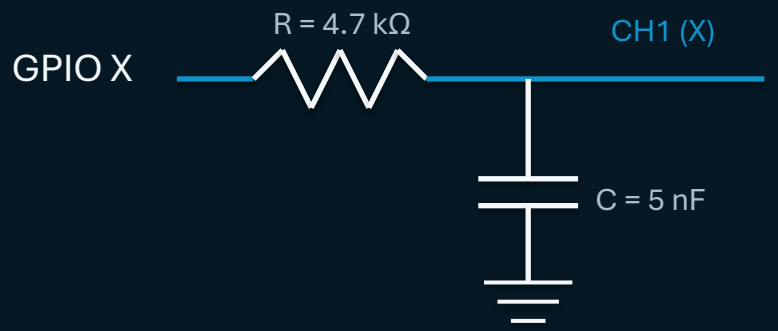
$$V(t) = V_{final} + (V_{initial} - V_{final})e^{-\frac{t}{RC}}$$



How an RC filter creates intermediate XY positions

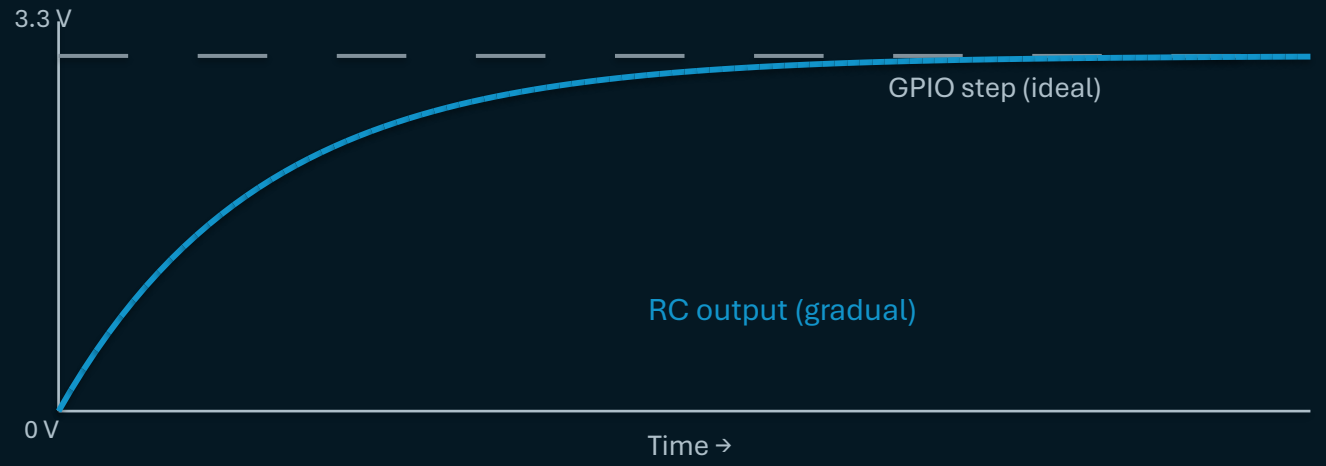
The capacitor charges and discharges gradually; the oscilloscope follows the resulting analog voltage.

The actual electrical circuit



Repeat the same circuit for GPIO Y → CH2.

Voltage response to a digital transition



Time constant

$$\tau = R \times C = 4700\ \Omega \times 5\ \text{nF} = 23.5\ \mu\text{s}$$

The RC filter smooths PWM into intermediate voltages. It does not draw a straight line by itself: the X/Y waveform, filtering and timing determine the visible path.

For a vector display, both axes need smoothly controlled voltages. Too much RC filtering rounds corners and slows transitions.

RC FILTER - CHOOSE R AND C FROM AVAILABLE COMPONENTS

The time constant τ sets smoothing and response speed. Match τ ≈ 23.5 µs, not necessarily the exact components.

4.7 kΩ

Green borders: approximately ±8% from 23.5 µs

TIME CONSTANT τ = R × C (values in µs)

R ↓ / C →	1 nF	2.2 nF	3.3 nF	4.7 nF	5 nF	6.8 nF	10 nF	22 nF
1 kΩ	1	2.2	3.3	4.7	5	6.8	10	**22**
2.2 kΩ	2.2	4.8	7.3	10.3	11	15	**22**	48.4
3.3 kΩ	3.3	7.3	10.9	15.5	16.5	**22.4**	33	72.6
4.7 kΩ	4.7	10.3	15.5	**22.1**	**23.5**	32	47	103.4
6.8 kΩ	6.8	15	**22.4**	32	34	46.2	68	149.6
10 kΩ	10	**22**	33	47	50	68	100	220
15 kΩ	15	33	49.5	70.5	75	102	150	330
22 kΩ	**22**	48.4	72.6	103.4	110	149.6	220	484

REFERENCE CIRCUIT

R = 4.7 kΩ C = 5 nF

τ = 23.5 µs = 0.0235 ms

PRACTICAL ALTERNATIVES

2.2 kΩ + 10 nF → 22 µs

3.3 kΩ + 6.8 nF → 22.4 µs

6.8 kΩ + 3.3 nF → 22.4 µs

10 kΩ + 2.2 nF → 22 µs

Smaller τ: faster, more ripple.
Larger τ: smoother, more rounding.

τ[µs] = R[kΩ] × C[nF] | τ[ms] = τ[µs] / 1000 | $f_c \approx 1 / (2\pi\tau)$ ≈ 6.8 kHz for 23.5 µs

What would happen without the RC filter?

- The pin changes digital levels millions of times per second.
- Voltage no longer represents the desired coordinate steadily.
- Traces may become very thick, irregular, or difficult to recognize.
- The filter averages pulses, allowing the spot to follow the vector path.

Clock, point rate, and image refresh

```
f_clk = 50 MHz ⇒ T_clk = 20 ns

if (timer == 1388) ...
→ update every 1389 clock ticks
→ 1389 × 20 ns = 27.78 µs / step

t = 0 ... 63 ⇒ 64 points per segment
```

Rapidly redrawing the entire outline makes the image appear stable.

S1 and S2: choosing the character

P → A → U → L → ♥ → P

- S1 (PIN_88): increment phase, wrapping 4 → 0.
- S2 (PIN_89): decrement phase, wrapping 0 → 4.
- Two registers synchronize each button to the FPGA clock.
- ~10 ms debounce: one physical press generates one event.

Buzzer: why is its idle state HIGH?

```
// OMDAZZ buzzer: HIGH when idle
output reg buzzer = 1'b1;

// On press: 4,000,000 clock cycles ≈ 80 ms
beep_count <= 4000000;

// Toggle every 12,500 clocks → ~2 kHz
if (tone_count == 12499) buzzer <= ~buzzer;

// When finished: buzzer <= 1'b1;
```

Experimentally, an idle LOW caused continuous sound; idle HIGH eliminated it.

How is the Verilog program organized?

1. Shape ROM	definitions of the P, A, U, L, and heart segments
2. Interpolation	computes x(t), y(t) between segment endpoints
3. Sigma-delta X/Y	converts coordinates into pulse densities
4. Timer	advances t and segment every 27.78 µs
5. Buttons + sound	debounced selection and 80 ms beep

Laboratory observations and limitations

- Square and triangle were displayed; diagonals may curve slightly.
- The circle was approximate and slightly flattened; more samples would help.
- The letter A became clear after proportional coordinate interpolation.
- Open paths such as U and L must be retraced to their start; otherwise a visible return trace appears.
- Calculated voltages are ideal averages; verify the actual amplitudes and XY settings.

Practical exercises for students

- Change (80,210) → (130,210) to (80,210) → (130,120). What line appears?
- Estimate voltage for coordinate X = 200 with 3.3 V I/O.
- Explain what happens to the capacitor as HIGH pulses become more frequent.
- Calculate the new τ if R is doubled while C remains 5 nF.
- Modify L's points to turn it into the letter T.
- Explain why two separate letters may require Z-axis blanking.

Conclusion: complete causal chain

8 bits
desired position

→

Interpolator
consecutive points

→

ΣΔ
HIGH fraction

→

R + C
charge and voltage

→

CH1 + CH2
spot moves

Two digital GPIO pins became an analog vector-drawing system.

Appendix — full Verilog code (1–24/202)

Verified implementation; assign the physical pins in Quartus Pin Planner.

```
001  module OsciDesen (
002      input  wire clk,
003      input  wire S1, S2,
004      output wire X, Y,
005      output reg  buzzer = 1'b1
006  );
007
008  reg [7:0] x = 80, y = 60;
009  reg [9:0] sx = 0, sy = 0;
010  reg [10:0] timer = 0;
011  reg [2:0] phase = 0;       // 0=P, 1=A, 2=U, 3=L, 4=Heart
012  reg [3:0] segment = 0;
013  reg [6:0] t = 0;
014
015  // Button synchronization
016  reg s1a = 1, s1b = 1;
017  reg s2a = 1, s2b = 1;
018
019  // Debounce
020  reg [18:0] db1 = 0, db2 = 0;
021  reg k1 = 1, k2 = 1;
022  reg prev1 = 1, prev2 = 1;
023
024  // Beep: 2 kHz for 80 ms
```

Appendix — full Verilog code (25–48/202)

Verified implementation; assign the physical pins in Quartus Pin Planner.

```
025  reg [13:0] tone_count = 0;
026  reg [21:0] beep_count = 0;
027
028  // Coordinates: {x0, y0, x1, y1}
029  function [31:0] shape;
030      input [2:0] p;
031      input [3:0] s;
032      begin
033          shape = {8'd80,8'd60,8'd80,8'd60};
034          case (p)
035          // P
036          0: case(s)
037              0: shape={8'd80,8'd60,8'd80,8'd210};
038              1: shape={8'd80,8'd210,8'd130,8'd210};
039              2: shape={8'd130,8'd210,8'd155,8'd190};
040              3: shape={8'd155,8'd190,8'd155,8'd160};
041              4: shape={8'd155,8'd160,8'd130,8'd140};
042              5: shape={8'd130,8'd140,8'd80,8'd140};
043              6: shape={8'd80,8'd140,8'd80,8'd60};
044          endcase
045          // A
046          1: case(s)
047              0: shape={8'd70,8'd60,8'd128,8'd210};
048              1: shape={8'd128,8'd210,8'd186,8'd60};
```

Appendix — full Verilog code (49–72/202)

Verified implementation; assign the physical pins in Quartus Pin Planner.

```
049             2: shape={8'd186,8'd60,8'd157,8'd135};
050             3: shape={8'd157,8'd135,8'd99,8'd135};
051             4: shape={8'd99,8'd135,8'd128,8'd210};
052             5: shape={8'd128,8'd210,8'd70,8'd60};
053         endcase
054         // U
055         2: case(s)
056             0: shape={8'd85,8'd210,8'd85,8'd90};
057             1: shape={8'd85,8'd90,8'd100,8'd60};
058             2: shape={8'd100,8'd60,8'd150,8'd60};
059             3: shape={8'd150,8'd60,8'd165,8'd90};
060             4: shape={8'd165,8'd90,8'd165,8'd210};
061             5: shape={8'd165,8'd210,8'd165,8'd90};
062             6: shape={8'd165,8'd90,8'd150,8'd60};
063             7: shape={8'd150,8'd60,8'd100,8'd60};
064             8: shape={8'd100,8'd60,8'd85,8'd90};
065             9: shape={8'd85,8'd90,8'd85,8'd210};
066         endcase
067         // L
068         3: case(s)
069             0: shape={8'd95,8'd210,8'd95,8'd60};
070             1: shape={8'd95,8'd60,8'd165,8'd60};
071             2: shape={8'd165,8'd60,8'd95,8'd60};
072             3: shape={8'd95,8'd60,8'd95,8'd210};
```

Appendix — full Verilog code (73–96/202)

Verified implementation; assign the physical pins in Quartus Pin Planner.

```
073         endcase
074         // Heart
075         4: case(s)
076             0: shape={8'd128,8'd55,8'd92,8'd95};
077             1: shape={8'd92,8'd95,8'd72,8'd140};
078             2: shape={8'd72,8'd140,8'd88,8'd180};
079             3: shape={8'd88,8'd180,8'd112,8'd195};
080             4: shape={8'd112,8'd195,8'd128,8'd175};
081             5: shape={8'd128,8'd175,8'd144,8'd195};
082             6: shape={8'd144,8'd195,8'd168,8'd180};
083             7: shape={8'd168,8'd180,8'd184,8'd140};
084             8: shape={8'd184,8'd140,8'd164,8'd95};
085             9: shape={8'd164,8'd95,8'd128,8'd55};
086         endcase
087         endcase
088     end
089 endfunction
090
091 reg [3:0] last_segment;
092 always @(*) begin
093     case (phase)
094         0: last_segment = 6;
095         1: last_segment = 5;
096         2: last_segment = 9;
```

Appendix — full Verilog code (97–120/202)

Verified implementation; assign the physical pins in Quartus Pin Planner.

```
097          3: last_segment = 3;
098          4: last_segment = 9;
099          default: last_segment = 0;
100      endcase
101  end
102
103  wire [31:0] v = shape(phase, segment);
104  wire [7:0] x0 = v[31:24];
105  wire [7:0] y0 = v[23:16];
106  wire [7:0] x1 = v[15:8];
107  wire [7:0] y1 = v[7:0];
108
109  wire signed [10:0] dx =
110      $signed({1'b0,x1}) - $signed({1'b0,x0});
111  wire signed [10:0] dy =
112      $signed({1'b0,y1}) - $signed({1'b0,y0});
113  wire signed [20:0] ix =
114      $signed({1'b0,x0}) +
115      (dx * $signed({1'b0,t})) / 63;
116  wire signed [20:0] iy =
117      $signed({1'b0,y0}) +
118      (dy * $signed({1'b0,t})) / 63;
119
120  wire press1 = prev1 && !k1;
```

Appendix — full Verilog code (121–144/202)

Verified implementation; assign the physical pins in Quartus Pin Planner.

```
121  wire press2 = prev2 && !k2;
122
123  always @(posedge clk) begin
124      // Synchronize buttons
125      s1a <= S1;
126      s1b <= s1a;
127      s2a <= S2;
128      s2b <= s2a;
129
130      // Debounce: approximately 10 ms
131      if (s1b == k1)
132          db1 <= 0;
133      else if (db1 == 499999) begin
134          db1 <= 0;
135          k1 <= s1b;
136      end else
137          db1 <= db1 + 1'b1;
138
139      if (s2b == k2)
140          db2 <= 0;
141      else if (db2 == 499999) begin
142          db2 <= 0;
143          k2 <= s2b;
144      end else
```

Appendix — full Verilog code (145–168/202)

Verified implementation; assign the physical pins in Quartus Pin Planner.

```
145          db2 <= db2 + 1'b1;
146
147      prev1 <= k1;
148      prev2 <= k2;
149
150      // Working sigma-delta DAC
151      sx <= {2'b00,sx[7:0]} + {2'b00,x};
152      sy <= {2'b00,sy[7:0]} + {2'b00,y};
153
154      // Button selection
155      if (press1 || press2) begin
156          if (press1)
157              phase <= (phase == 4) ? 0 : phase + 1'b1;
158          else
159              phase <= (phase == 0) ? 4 : phase - 1'b1;
160
161          segment <= 0;
162          t <= 0;
163          timer <= 0;
164
165          // Start beep
166          beep_count <= 4000000;
167          tone_count <= 0;
168          buzzer <= 1'b1;
```

Appendix — full Verilog code (169–192/202)

Verified implementation; assign the physical pins in Quartus Pin Planner.

```
169      end else begin
170          // Drawing engine
171          if (timer == 1388) begin
172              timer <= 0;
173              x <= ix[7:0];
174              y <= iy[7:0];
175              if (t == 63) begin
176                  t <= 0;
177                  segment <= (segment == last_segment) ?
178                             0 : segment + 1'b1;
179              end else
180                  t <= t + 1'b1;
181          end else
182              timer <= timer + 1'b1;
183
184          // Buzzer oscillator
185          if (beep_count != 0) begin
186              beep_count <= beep_count - 1'b1;
187              if (tone_count == 12499) begin
188                  tone_count <= 0;
189                  buzzer <= ~buzzer;
190              end else
191                  tone_count <= tone_count + 1'b1;
192          end else begin
```

Appendix — full Verilog code (193–202/202)

Verified implementation; assign the physical pins in Quartus Pin Planner.

```
193             // Inactive buzzer = HIGH
194             buzzer <= 1'b1;
195             tone_count <= 0;
196         end
197     end
198 end
199 
200 assign X = sx[8];
201 assign Y = sy[8];
202 endmodule
```

Results

